

Final Report: Performance of region-based and pixel-based loss functions in martian terrain segmentation for navigation using the AI4Mars dataset

Group 17:

Darlington Nkrumah (202492437)

Greg de Souza (202582225)

Xuan Toan Doan (202583882)

abstract This report presents a comparative study of pixel-based and region-based loss functions in semantic segmentation on the navigational dataset for Mars Mission m2020, available in the open AI4Mars dataset. We implement torch's DeepLabV3+ with a Resnet-50 backbone, using transfer learning techniques from the default weight, and from a pre-training process on the larger MSL navigational dataset. By implemented weighted sampling and weighting the loss functions around the underrepresented big rock class, we are able to achieve accuracy, recall and IoU comparable to other related work. The final assessment shows that region-based loss functions seem to perform worse than pixel-based loss function on the m2020 dataset, with the best log cosh dice loss achieving around 60% mIoU, and the best cross entropy based model achieving 64% mIoU.

I. INTRODUCTION

Semantic segmentation is a computer vision tool with the task of assigning a semantic label to each pixel of an image, or in other terms, defining boundaries between different semantic classes [1]. Semantic segmentation has been successfully applied in numerous different contexts, from medical imaging to self-driving cars [2], [3].

In 2020 NASA has published an open dataset of images with segmentation masks from pictures taken by martian rovers as part of the MSL and M2020 missions [4]. Numerous groups have evaluated the capacity of different neural networks in executing semantic segmentation tasks on the AI4Mars and similar dataset, including access time to prediction in the limited hardware of the rovers [5], [6], [7].

In this work we focus our effort into evaluating the choice of loss functions in image segmentation tasks in the m2020 navigational dataset from the AI4Mars dataset, containing 2355 images and masks, comparing the effectiveness of a pixel-based loss function - weighted cross entropy (WCE) - and a region-based loss function - weighted log cosh dice loss (LCDL) - in segmenting the navigational dataset from m2020 mars mission. We also explore the effects of pre-training the model of choice with the MSL Navigational

dataset, containing 16396 labeled images, a retraining strategy that provided mixed results in previous works [4].

The model chosen was the DeepLabv3+ with a ResNet-50 backbone, a model that provides time to prediction of 25ms and mean accuracy of 87% when trained and tested in a similar dataset of martian terrain images by Luna et al [7] in an independent dataset of Mars terrain images. We apply this model to the open AI4Mars dataset to evaluate if the pre training with the MSL and the choice of loss functions can improve the recall and mean Intersection over Union (mIoU) results in comparison to each other, and related work.

In section 2 we review relevant previous work, both on categorizing different models for semantic segmentation, as well as previous results of other attempts to do image segmentation on the AI4Mars or similar independent datasets. In section 3 we present our strategy to process the data and train the model with two different loss functions - Cross entropy and Log Cosh Dice Loss - and with and without pre training on the MSL dataset. In section 4, we present the results of our work and discuss them in relation to other relevant works.

II. LITERATURE REVIEW

In this section we review two sets of related works - The first review concepts, models and choice of loss functions in the context of semantic segmentation to motivate our choices in building our model. In the second subsection we review results of previous related works around their AI4Mars and similar datasets.

A. Semantic Segmentation

1) *Models*: Image segmentation is an image processing technique used to divide an image into two or more meaningful regions, or alternatively, the process of defining boundaries separating semantic entities in an image. In a more technical definition, the task of image segmentation is to assign a label to each pixel within an image, with each label connected with respect to some property [8].

Image segmentation can be divided in three categories: Instance segmentation, in which different instances of the

same semantic label are categorized independently; semantic segmentation, which assigns different labels for regions with different semantic meaning without classifying independent instances independently; and panoptic segmentation, which performs both tasks simultaneously [3].

The task of image segmentation has been significantly impacted by the development and advancement of deep learning techniques in computer vision, in particular after the success of AlexNet in 2012 in using CNN for Semantic Segmentation. Since then, numerous Deep Learning models for semantic segmentation have been developed with significant success [3], [1], [9].

Some general classes of models for semantic segmentation include Convolutional Neural Networks (CNNs), which apply kernel-based convolutions to capture the positional relations of features in images. By adding pooling and unpooling layers that change the original size, it allows CNNs to find meaningful information at multiple scales of the feature image by following a “pyramid-like” design. CNNs are among the most successful approaches to semantic segmentation [9].

Another important advancement in machine vision was the development of Residual Neural Networks [10], developed as a tool to mitigate the degradation that occurred with large neural networks by adding the output of earlier layers to the input of deeper layers of the network, effectively “skipping” some layers. Residual blocks allow for deeper networks that do not suffer as much from degradation and vanishing coefficients as pure convolutional neural networks.

Another relevant advance in convolutional neural networks is the idea of dilated (or “atrous”) convolutions [9], [11], which add an additional parameter to convolutions that increases the perceptive field without adding new parameters, according to the dilation rate r , shown in equation 1, where x represents the input and w represents the kernel. A visual representation of the effects of the dilution rate in a 3×3 kernel is shown in figure 1. In practice, the dilution rate makes the convolution pick samples farther from the center of the kernel, “skipping” neighbors, this increases the field of view without increasing the Kernel size.

$$y_i = \sum_{k=1}^K x[i + rk]w[k] \quad (1)$$

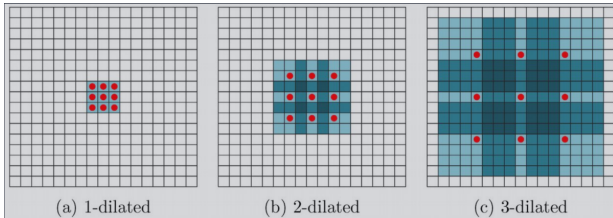


Fig. 1: Visual representation of dilated convolution with a 3×3 Kernel at different dilation rates. [9]

The basis for the DeepLab family of models are a combination of dilated convolutions, a pyramid network

for encoding the original input, and a CNN for decoding the final pyramid layer. This combination of convolution layers and residual blocks to extract the features of the original image in multiple scales and stack them together in a multichannel output. In traditional pyramid networks, this final layer is already close to the output as shown in figure 2. The DeepLab model adds complexity by using the pyramid-like section of its design to just encode the information of the original input, the encoded features are then extracted by another CNN structure from the multi-channel pyramid output, as represented in figure 3.

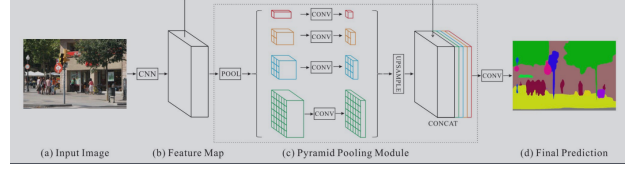


Fig. 2: The PSPN architecture. A CNN produces the feature map and a pyramid pooling module aggregates the different sub-region representations. Up-sampling and concatenation are used to form the final feature representation from which, the final pixel-wise prediction is obtained through convolution [9].

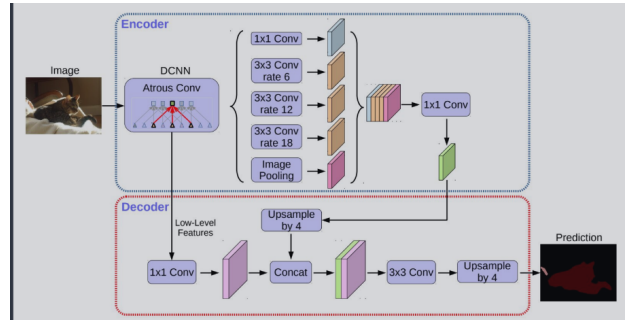


Fig. 3: The DeepLabV3+ model. From [9].

2) *Loss Functions:* Another important choice in implementing a deep learning model is the choice of the loss function, a metric that evaluates how far a given output is from the expected ground truth. A good choice of loss function can mitigate usual challenges of image vision tasks, like noisy datasets and unbalanced class representation.

A number of loss functions are presented and evaluated by the work done in [2]. Here we present the most relevant loss functions for our attempt at Semantic Segmentation of the A14Mars Dataset.

One of the most common loss metrics is the Cross-entropy (CE) loss function. It measures the difference between two probability distributions given a random variable. In segmentation tasks, this means comparing the ground truth with the assigned probability that the pixel is of that class according to the model (i.e. output of the soft max function). The formula for the cross entropy is based on Shannon’s entropy, and is presented in a practical formulation in equation 2 , t_n is a one-hot encoded vector that is 1 on the target class and zero on the others, and $\hat{y}$ is the predicted probability.

In imbalanced datasets, one common approach to minimize the effects of the imbalance in the training of the model is adding weights that favor the less represented classes. If w is a vector of the weights of each labeled class, the weighted cross entropy is presented in equation 3.

$$L_{CE} = - \sum_{i=1}^C \log(t_i \cdot \hat{y}_i) \quad (2)$$

$$L_{CE} = - \sum_{i=1}^C t_i \cdot w \log(t_i \cdot \hat{y}_i) \quad (3)$$

Cross-entropy loss is part of the Pixel level loss functions, this family of loss functions evaluate each individual pixel as a part of their computation for dissimilarity, and they usually improve pixel-based assessment. Another approach are region-level loss functions, that focus on scoring based on the capacity of a model to adequately separate different semantic regions of an image. One example of region-level loss function is the Dice Loss function, that scores similarity based on the intersection between the predicted subset of pixels Y and the ground truth region T , and is written as in equation 4 for two class classification task, where $|A|$ represents the size of a set A . Form multi class segmentation tasks, the formula is presented in equation 5, where $\hat{y}$ represent the prediction probability of each class, and t is the vector that represents the ground truth. There is a choice in the number of classes considered in the context of a possible background/negative class, this background class is sometimes excluded from loss computations.

$$L_{Dice} = 1 - \frac{2|Y \cap T|}{|Y| + |T|} \quad (4)$$

$$L_{Dice} = 1 - \frac{1}{C} \sum_{c=1}^C \frac{2 \sum_i t_{ic} \hat{y}_{ic}}{\sum_i t_{ic} + \sum_i \hat{y}_{ic}} \quad (5)$$

A variation of the Dice Loss that offer some advantages in segmentation tasks is the log cosh Dice Loss, shown in equation 6, it enhances smoothness and robustness to outliers, and minimizes the effects of outliers, and it's a better behaved function in terms of derivatives, that helps the optimization steps. The Dice Loss can also be "weighted" in a sense. A generalized version of the dice loss is defined by the Generalized Wasserstein Dice Loss, the full scope of the generalization is outside of this work scope, but a less general version of weighted dice loss was used in one of the training steps, presented in subsection III-B.

$$L_{\log-\cosh} = \log(\cosh(L_{Dice})) \quad (6)$$

B. Martian Terrain Segmentation Related Work

The task of using computer vision for autonomous driving capability has been explored by NASA since 2004 [4] with the AutoNav project. The advancements in computer vision through deep learning models presented a great opportunity to improve the autonomous driving capabilities of the two

active Martian rovers: Curiosity, launched as part of the Mars Science Laboratory (MSL) mission, and Perseverance, launched as part of the Mars 2020 (m2020) mission. Since advancements in computer vision fueled by development of deep-learning techniques, several research groups have attempted to use pictures from martian images, both from rovers and satellites, as input in segmentation tasks, with goal of identifying meaningful geological features or terrain for navigation. [4], [6], [5], [7]. The AI4Mars dataset is just of the datasets explored by this groups, while others have independent dataset, or have labeled the available images through different methods, or even implemented some level of semi-supervised learning algorithms [5].

The AI4Mars navigational dataset divides martian terrain in four classes [4] :

- **Soil:** Good for navigation
- **Bedrock:** Ideal for navigation.
- **Sand:** A less favorable but still viable path choice.
- **Big Rock:** Must be avoided while choosing a path.

The AI4Mars dataset is a publicly available set of images and semantic segmentation masks developed through a citizen science initiative by NASA. A number of volunteers were trained to create the masks, and their work was reviewed by specialist laborers and condensed into the final dataset [4]. One notable challenge in the dataset is that the Big Rock class is underrepresented, consisting of less than 2% of pixels in the m2020 dataset, as shown in Figure 4.

The work by Swan et al [4] trained a DeepLabV3+ with ResNet-101 on two versions of the MSL dataset, created by two different labeling strategies called *random*, that used one possible mask from multiple suggested by trained labeler, and the *merged* strategy, where the final masks was a merged version of all suggestion by trained labelers, compiled by an expert on martian terrain. Their work showed that the model performed significantly better with the *merged* masks, with the accuracy for big rocks increasing from 82.83% to 93.24%. The final confusion matrices from this work are presented in table I

Actual	Predicted			
	Soil	Bedrock	Sand	Big Rock
<i>MSL Random (Overall Accuracy: 94.97%)</i>				
Soil	96.00	0.31	3.69	0.00
Bedrock	6.15	90.87	2.54	0.44
Sand	0.25	3.23	96.51	0.01
Big Rock	11.67	0.03	5.48	82.83
<i>MSL Merged (Overall Accuracy: 96.67%)</i>				
Soil	99.10	0.32	0.57	0.01
Bedrock	3.64	94.90	0.37	1.09
Sand	0.88	5.62	93.45	0.05
Big Rock	6.76	0.00	0.00	93.24

TABLE I: MSL NAVCAM Confusion Matrix Percentages for *random* and *merged* labeling strategies from Swan et al [4]

The work by Atha et Al [6] explores additional machine learning models for semantic segmentation of martian terrain in the context navigation, exploring how different models

perform both on the MSL and on the M2020 navigational dataset. Their work identified two main problems with current models that aim to allow for independent path making for martian rovers. First, models tend to perform worse as the rovers move, and terrain around them changes naturally, the current models aren't general enough for the martian surface. Second, the accuracy necessary for a model to perform independently, that is, without a remote human operator offering input, is significantly higher than the accuracy obtained by models at the time of that work.

For their model, they trained a ResNet-101 from zero with both MSL and M2020 datasets, they also trained a ResNet-101 from the weights for ImageNet, and they also tried to transfer train the model trained on the larger MSL and MER datasets to the M2020 dataset. The transfer learning did improve performance over soil and rock, but decreased accuracy on labeling bedrock. Their results in relation to the m2020 Dataset are presented in table II.

Luna et al [7] implemented ViBEKO's DeepLabV3+ as part of their efforts to develop FASTNAV, their tool for independent rover navigation. The dataset used was independently obtained and labeled by them. Their implementation of DeepLabV3+ with ResNet-50 as a backbone achieved 87% accuracy and 72% mIoU with a time to prediction of 25ms, relevant for practical use in a rover.

III. TRAINING METHOD

A. Dataset and Preprocessing

The m2020 navigational dataset, consisting of 2355 images with class label masks. The masks were produced by a crowd science initiative that trained labelers to identify the four classes: soil, bedrock, sand and big rocks, represented by pixel color values of 0,1,2,3 respectively. The masks produced by the trained labelers was reviewed and finalized by a group of expert labelers to produce the final dataset. The images and masks aren't always the same size, some images were centercropped before the labels were constructed, for this images it's necessary to center crop the images accordingly when retrieving them from the dataset [4].

For training, The dataset is split randomly into 90% for the training set and 10% for the validation set. Both datasets are resized, grayscaled, and converted into normalized tensors using the mean and variation suggested by the PyTorch implementation of the DeepLabV3+ model [12], with the mean by RGB channel as $\text{mean}=[0.485, 0.456, 0.406]$, with standard deviation of $[0.229, 0.224, 0.225]$.

The training dataset additionally passes through an augmentation loop which includes:

- Horizontal flip (50% probability): The idea here is that the rover can take pictures of the same area viewed from different sides, and the model should still be able to perform accordingly.
- Rotation up to 20° (50% probability): Some of the rover cameras are free moving and may be rotate, so a generalized understanding of the geometry of the classes is important.

- Random cutouts (up to 8, 50% probability): Mostly to avoid over training on noise due to the limited dataset size
- Random brightness adjustment (20% probability): pictures are taken under different light conditions in a changing environment like the martian surface.

One recurring problem in related works [6], [5] was that big rocks are underrepresented, consisting of only 3.2% of non-background pixels as shown in figure 4, to compensate a little bit for this during training, the data-loader was set with a weighted sampler that favored loading images with big rock pixels in their mask by a factor of 15.

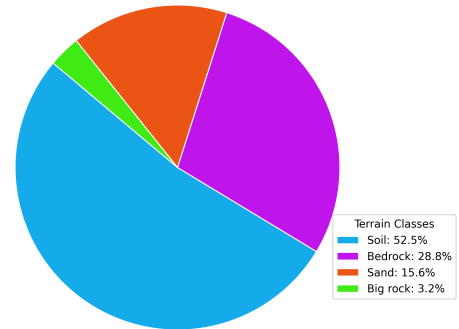


Fig. 4: Relative abundance of each non-background class in the AI4Mars m2020 navigational dataset.

B. Model Training: DeepLabV3+ with ResNet-50

DeepLabV3+ belongs to a family of neural networks that exploits dilated (atrous) convolutions to increase the field of view without increasing the number of parameters. While Luna et al [7] was successful in implementing DeepLabV3+ on an independent dataset of Martian images, this work implements DeepLabV3+ with a ResNet-50 backbone on the open AI4Mars dataset.

We utilized the PyTorch implementation of DeepLabV3+, which employs Atrous Spatial Pyramid Pooling (ASPP) with an encoder-decoder topology [11]. The training process uses different initial learning rates: 0.0005 for the classifier (ASPP onwards) and 0.00001 for the ResNet-50 backbone. The low learning rates chosen try to avoid deviating too much from the base model during transfer learning, since the number of images is relatively low, we are trying to fine tune the default weights of the model for a new problem, since it's unlikely we can train a model from zero with so few images.

The model was trained with two possible choices of loss function: weighted Cross Entropy Loss, and weighted Log-Cosh Dice Loss. The cross entropy loss scores the model in it's ability to correctly label each pixel, while the Log-Cosh Dice Loss scores based on the ability to adequately define the correct boundaries of a region. The choice of the log cosh version of dice loss is due to the big rock class often being present in just a small section of the image, resulting in fluctuating values for the pure dice loss. The choice of

log-cosh dice loss makes the dice loss a bit more resistant to these variations [2] The weighted cross entropy formula is shown in equation 7, $w_c \in 0.04, 0.08, 0.15, 0.73$. The log cosh dice loss formula is shown in equation 8 using the same weights. The big rock class was assigned the dominant weight to prevent the gradient from being overwhelmed by the more abundant classes.

$$L_{WCE} = - \sum_{c=1}^C w_c \cdot y_c \log(\hat{y}_c) \quad (7)$$

$$L_{WLCDL} = \log(\cosh(1 - \sum_{c=1}^C w_c \frac{2 \sum_i y_{ic} \hat{y}_{ic}}{\sum_i y_{ic} + \sum_i \hat{y}_{ic}})) \quad (8)$$

The loss functions were also weighted inversely with the abundance of each class, according to figure 4, so the soil class was weighted as for 4% of the final loss value, 8% for bedrock, 15% for sand and 73% for big rock. The loss functions also ignored the background, which in retrospect was a mistake, the final models learned to always guess a class (usually soil) to minimize their loss, since “missing” the background isn’t penalized, as shown in figures 7 and 8.

For each choice of loss function, the model either trained directly from the default weights of the torch implementation of DeepLabV3+, or it was initialized from the default weights, then pre trained on the 16396 images of the MSL dataset and then trained and assessed on the m2020 datasets. This training process resulted in four different final models with different scores

The optimizer is Stochastic Gradient Descent (SGD) with a decay rate of 0.0001 and momentum of 0.9. A polynomial scheduler is used to dynamically decrease the learning rate with a power of 0.9.

The model is trained for a total of 150 epochs using a progressive resizing strategy:

- **Epochs 1–50:** 128×128 pixels resolution, batch size of 16, with two accumulation steps before zeroing the gradient.
- **Epochs 51–100:** 256×256 pixels resolution, batch size of 8, with 4 accumulation steps
- **Epochs 101–150:** 512×512 pixels resolution, batch size of 4 with 4 accumulation steps.

In the last training loop, the model was saved only if the beat the mIoU of the previously saved model, this early saving was an attempt to minimize over-training, but except for the model trained directly with cross entropy loss and no pre-training, that performed the best at epoch 34, all other models were saved around epoch 145-150, suggesting those models could have performed even better given more training time. The resizing strategy of increasing the image size every 50 epochs was due to the limited time to train the model, the training time with larger images was taking a significant while, so that was a compromise to make this project deliverable on time. It’s possible that fully training

the model on larger images from that start might provide better results.

IV. RESULTS

The evolution of the assessment on accuracy and mean intersection over union (mIoU) of the four versions of the models with different trained set ups are presented in figures 5 and 6. The choice of mIoU for model selection is due to accuracy being a misleading metric over unbalanced classes. A model guessing huge portions of Big Rocks would get high accuracy on big rocks by chance. The mIoU assessment metric is better at assessing the model’s ability to in fact define the boundaries between the semantic relevant classes.

Regarding the resizing between training epochs, this choice seems to have affected the Log Likelihood Dice Loss (LCDL) trained models negatively, with the models performing worse for a significant number of epochs after the resizing, this makes sense for a region-based loss function, that “cares” more about the overall structure of the regions. For the cross-entropy loss trained models, the model seems to quickly adapt to the resized images and improve their total accuracy and mIoU due to the larger images.

Pre-training on the MSL dataset didn’t significantly improve the accuracy and mIoU of the cross entropy based model, with the model achieving an accuracy of around 80.4% and mIoU of 64% regardless of pretraining, this is consistent with the finding of previous works on the AI4Mars dataset [6], [5]. The findings for the log cosh Dice Loss trained model is similar in the overall assessment, with maximum accuracy of around 70.7% and mIoU of 60% regardless of pretraining.

On the ability to classify specific classes, the cross entropy loss trained models seem to outperform the LCDL model significantly in the underrepresented big rock class, achieving around 74% accuracy and recall on that class, against 57% on these metrics for the best performing dice loss based models. In the dice loss based models, pretraining hurt the ability of the model in identifying big rocks, with the accuracy reducing from 57% to 49% with pretraining.

It’s important to also note a problem in the choice of ignoring the background when computing these losses, due to this choice, the model learned to ALWAYS guess a class for each pixel, as shown in figures 7 and 8. The prediction doesn’t contain any background prediction (colored white on the ground truth), the model was overconfident that anything could be classified as one terrain, so it even included the sky as a type of soil.

The work by Luna et al [7] also trained the DeepLabV3+ with ResNet-50 as a backbone on an independent but similar dataset to the AI4Mars dataset. Their model achieved 87% accuracy and 72% mIoU. It’s not likely we can know where their improvements can from, if it was due to better choice of loss function and training hyperparameters, or if it was due to the different dataset.

The results per class assessed on Recall and IoU are presented in table III. The work by Atha et al [6] - presented on table II trained models with ResNet-101 backbone, serving

as a point of comparison for our results, specially in investigating the effects of the additional DeepLabV3+ layers. Our training methods seemed to have universally improved the prediction of soil, but that is most likely due to our model ignoring the loss on predicting the background, that resulted in the model always guessing something for every pixel, and soil was the most common class. The Recall results for the other classes also shown an improvement over the pure ResNet-101, and for some classes, and improvement over the ResNet-101 with pretraining. The ResNet-101 with ImageNet still outperforms DeepLabV3+ with ResNet-50 backbone on all assessed metrics.

Metric	Class	ResNet-101	+ImageNet	+MSL
Precision	Soil	0.933	0.999	0.916
	Bedrock	0.857	0.467	0.917
	Sand	0.664	0.667	0.754
	Bigrock	0.558	0.618	0.174
Recall	Soil	0.746	0.596	0.838
	Bedrock	0.570	0.958	0.536
	Sand	0.975	0.952	0.957
	Bigrock	0.283	0.355	0.640

TABLE II: Performance metrics for models trained on the M2020 dataset by Atha et al. [6], reorganized for single-column fit.

Metric	Class	CE	CE+PT	LCDL	LCDL+PT
Recall	Soil	0.871	0.864	0.899	0.923
	Bedrock	0.825	0.853	0.773	0.725
	Sand	0.811	0.762	0.587	0.695
	Bigrock	0.710	0.743	0.572	0.496
IoU	Soil	0.789	0.790	0.712	0.728
	Bedrock	0.725	0.753	0.683	0.651
	Sand	0.668	0.586	0.498	0.605
	Bigrock	0.412	0.443	0.490	0.434

TABLE III: Recall and IoU metrics for the m2020 validation dataset using DeepLabV3+ with ResNet-50 across various configurations. CE: Cross Entropy Loss, LCDL: Log Cosh Dice Loss, PT: Pretrained.

V. CONCLUSION

In this work we reviewed the literature on Semantic Segmentation tasks applied to martian terrain in the context of navigational dataset, such as the open AI4Mars Dataset provided by NASA [4] and similar datasets, such as the one explored by Luna et al [7]. A review on models and loss functions for semantic segmentation [2], [1], [3], [9] motivated our work in comparing the effect of a choice of pixel-level loss function (Cross Entropy) and a region-level based loss function (Log Cosh Dice Loss) on segmentation tasks for martian terrain using the DeepLabV3+ with ResNet-50 backbone.

While this model has been explored in the context of martian terrain segmentation [7], it was not applied to the open AI4Mars dataset. The work by Atha et al [6] implemented models with ResNet-101 backbone for AI4Mars segmentation, serving as a good comparison point for assessment scores. The AI4mars dataset contains two

sets of navigational images, from the M2020 mars mission and from the MSL mission. We trained our model do to image segmentation on the AI4Mars m2020 dataset, based on masks produced by a crowd science contributions and reviewed by expert labelers [4].

The model was trained using a resizing strategy with small batch sizes, but some accumulation steps for the gradient due to our choice of low learning rates to avoid deviate too much from the default weights of the model. The work produced four final models, based on the two choices of loss functions: weighted Cross entropy or weighted LCDL, both loss functions ignored the background, and the possibility of pre-training the model on the MSL dataset. Our best performing model overall was the pre-trained cross entropy loss model, achieving a mean accuracy of 80.5% and mIoU of 64%, with a recall on the underrepresented big rock class of 74.2%.

The final assessment result was lower but consistent with results by Luna et al [7], that achieved accuracy of 87% with mIoU of 72% with their independent navigational dataset. This lower performance might be due to differences in the dataset, but it also might be to some less than ideal choices. Further exploration on the exact weights for the weighted loss functions might have balanced the training better. Also, excluding the background from the loss function proved to be a mistake, resulting in models that are not capable of identifying when something is not terrain, as shown in figures 7 and 8.

The assessment results between the pixel-based loss function (Cross entropy) and region-based loss function (LCDL) suggest that pixel-based loss functions are better suited for segmentation tasks on navigational datasets of martian terrain, probably due to the fact that big rocks represent small regions, but are very significant for the segmentation tasks, making region-based loss functions less reliable.

REFERENCES

- [1] R. S. Kale and D. S. Thorat, "Image segmentation techniques with machine learning," *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 7, no. 6, p. 232–235, Dec. 2021.
- [2] R. Azad, M. Heidary, K. Yilmaz, M. Hüttemann, S. Karimijafarbigloo, Y. Wu, A. Schmeink, and D. Merhof, "Loss functions in the era of semantic segmentation: A survey and outlook," no. arXiv:2312.05391, Dec. 2023, arXiv:2312.05391 [cs]. [Online]. Available: <http://arxiv.org/abs/2312.05391>
- [3] F. Sultana, A. Sufian, and P. Dutta, "Evolution of image segmentation using deep convolutional neural network: A survey," *Knowledge-Based Systems*, vol. 201–202, p. 106062, Aug. 2020.
- [4] R. M. Swan, D. Atha, H. A. Leopold, M. Gildner, S. Oij, C. Chiu, and M. Ono, "Ai4mars: A dataset for terrain-aware autonomous driving on mars," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021, p. 1982–1991, iSSN: 2160-7516. [Online]. Available: <https://ieeexplore.ieee.org/document/9523149>
- [5] G. Goutham, H. Juneja, A. C, and V. R. B. Prasad, "Semantic segmentation on martian terrain for navigation using transformers," in *2022 IEEE 7th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, vol. 7, Dec. 2022, p. 276–282. [Online]. Available: <https://ieeexplore.ieee.org/document/10054341>

- [6] D. Atha, R. M. Swan, A. Didier, Z. Hasnain, and M. Ono, "Multi-mission terrain classifier for safe rover navigation and automated science," in *2022 IEEE Aerospace Conference (AERO)*, Mar. 2022, p. 1–13. [Online]. Available: <https://ieeexplore.ieee.org/document/9843615/>
- [7] C. Luna, R. Field, and S. Kay, "Ai-enabled capabilities to facilitate next-generation rover surface operations," no. arXiv:2510.05985, Oct. 2025, arXiv:2510.05985 [cs]. [Online]. Available: <http://arxiv.org/abs/2510.05985>
- [8] S. Ghosh, N. Das, I. Das, and U. Maulik, "Understanding deep learning techniques for image segmentation," no. arXiv:1907.06119, 2019, arXiv:1907.06119 [cs]. [Online]. Available: <http://arxiv.org/abs/1907.06119>
- [9] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," no. arXiv:2001.05566, Nov. 2020, arXiv:2001.05566 [cs]. [Online]. Available: <http://arxiv.org/abs/2001.05566>
- [10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Las Vegas, NV, USA: IEEE, 2016, p. 770–778. [Online]. Available: <http://ieeexplore.ieee.org/document/7780459/>
- [11] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," no. arXiv:1706.05587, Dec. 2017, arXiv:1706.05587 [cs]. [Online]. Available: <http://arxiv.org/abs/1706.05587>
- [12] P. Maintainers, "Deeplabv3_resnet50 weights documentation," https://pytorch.org/vision/stable/models/generated/torchvision.models.segmentation.deeplabv3_resnet50.html, 2026.

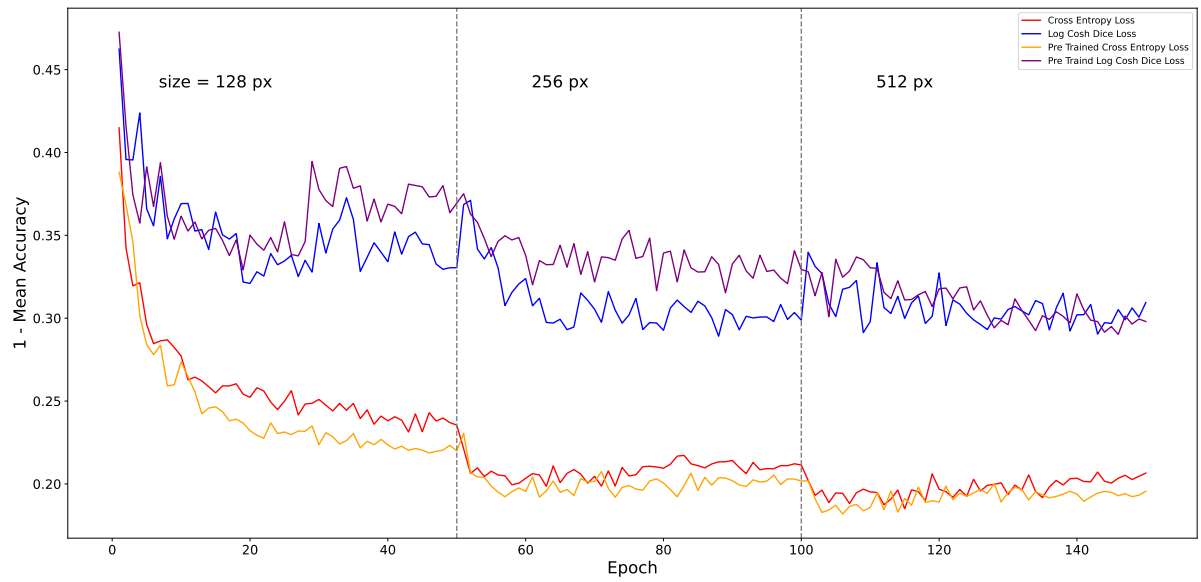


Fig. 5: Evolution of 1 - accuracy during training. At each 50 epochs, the model was trained with larger images

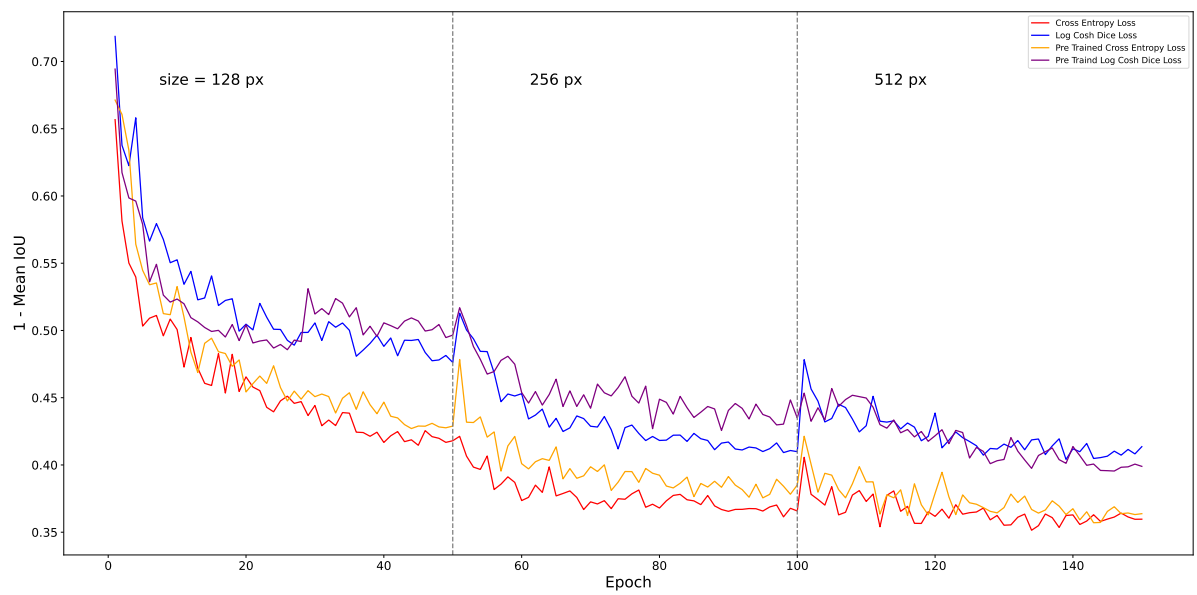


Fig. 6: Evolution of 1 - Mean Intersection over union (mIoU) during training. At each 50 epochs, the model was trained with larger images.

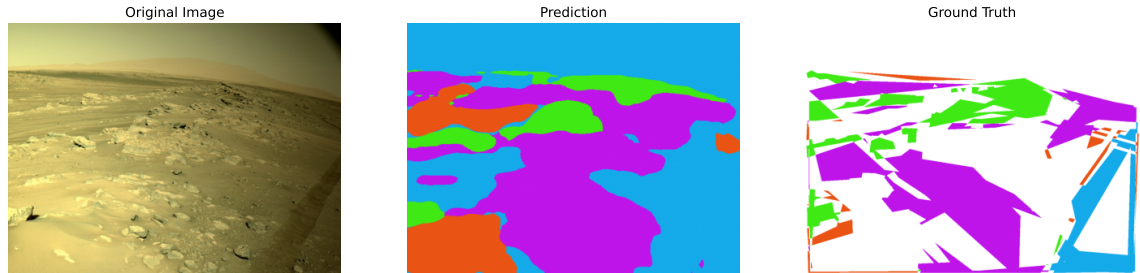


Fig. 7: Example of original image, model prediction and ground truth mask for the best performing model trained with the Log Cosh Dice Loss function. This model was pretrained on the MSL dataset. The label colors are: Blue for soil, Purple for Bedrock, Orange for Sand and Green for Big Rock

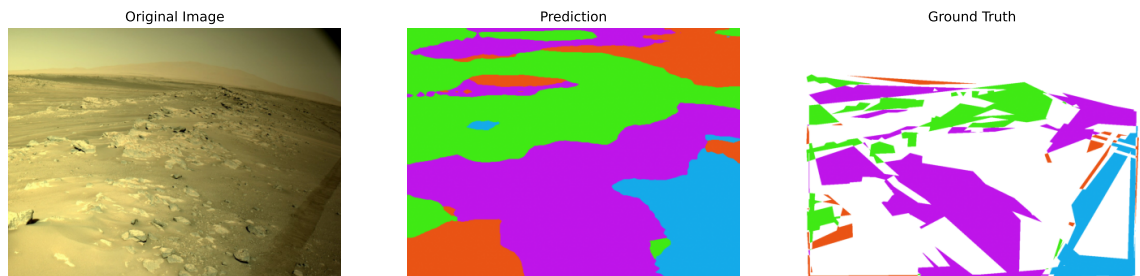


Fig. 8: Example of original image, model prediction and ground truth mask for the best performing model trained with the Cross Entropy Loss function. This model was pretrained on the MSL dataset. The label colors are: Blue for soil, Purple for Bedrock, Orange for Sand and Green for Big Rock